TEK306: Sikkerhet i mobile applikasjoner

Security in iOS Applications

Contents

- What is iOS?
- Understanding iOS Architecture
- iOS applications- categories, distribution, structure
- iOS Security Model- secure boot chain, secure enclave, code signing, sandboxing
- Data-at-rest encryption
- Exploit mitigation features
- More on security features- permission, API, keychain
- Reverse engineering, jailbreaking
- Writing Secure iOS Applications

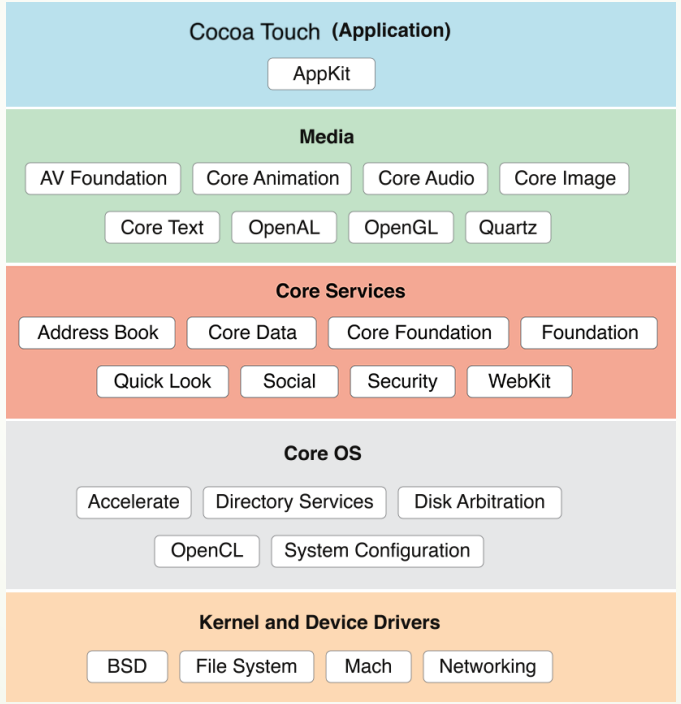
What is iOS?

- iPhone Operating System (**iOS**)- developed by Apple Inc. and distributed exclusively for **Apple hardware**.
- iOS is derived from **OSX-** which is also based on the **UNIX** operating system.
- **It powers-** iPhone, iPod, iPad, Apple watch, and Apple TV.
- Apple designed the iOS platform with **security at its core**.
- It has several versions the latest being **iOS 16**

iOS Architecture

- iOS has a layered architecture:
  - => Apps do not communicate to the underlying hardware directly.
  - **Core OS Layer-** consist of components that provide **low-level services-** hardware, network, kernel and device drivers.
  - **Core Services Layer-** consist services like WebKit, Address book, Security, Social and gives access to fundamental resources needed for app.
  - **Media Layer-** help you to use 2D and 3D **graphics**, animations, image effects, and professional-grade audio and **video** functionalities into mobile app.
  - **Cocoa Touch Layer- appearance of apps**, access to system functions like Contacts, Camera, **touch input**, shares with other apps, **push notifications** etc.

iOS Architecture …



Layered architecture

iOS applications- categories

- Standard native app- in Objective-C or Swift- also mixture of Objective-C, C, or C++.
- Browser-based app- mobile-friendly web app to render in iOS web views (e.g., MobileSafari):
  - Uses HTML, JavaScript, and CSS (and web security).
- Hybrid app- used with a native wrapper to display browser-based apps using a mobile web view.

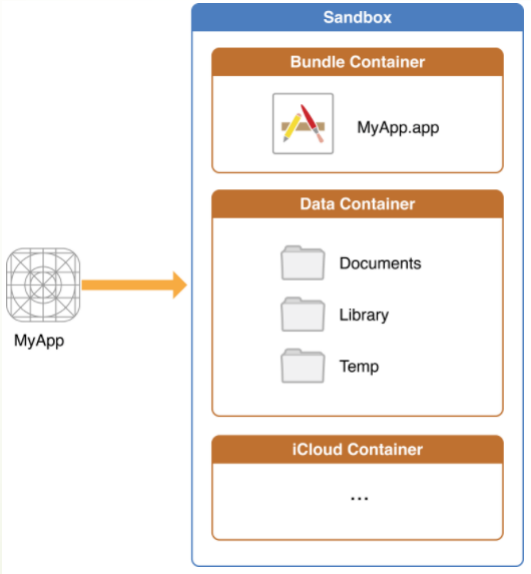
iOS Applications- distribution

- Apple App Store - is the official distribution market where users download iOS apps.
  - Before developers publish apps they must have an Apple developer account and own developer certificate.
- Enterprise distribution- iOS enterprise developer program allows organizations to develop and distribute in-house app to employees.

- **Why do enterprises employ their own distribution centers?**

iOS Applications- structure

- iOS apps are distributed as iOS App Store package (IPA) archive.
- It is a compressed package containing the necessary- compiled code, resources, and metadata required by a complete app.



iOS Applications- installing

- Method 1: How to Install IPA on iPhone with Online IPA Installer
- Method 2: Install IPA with Cydia Impactor- if file is on computer
- Method 3: Install IPA on iPhone Directly without Computer

- Details on development and testing.

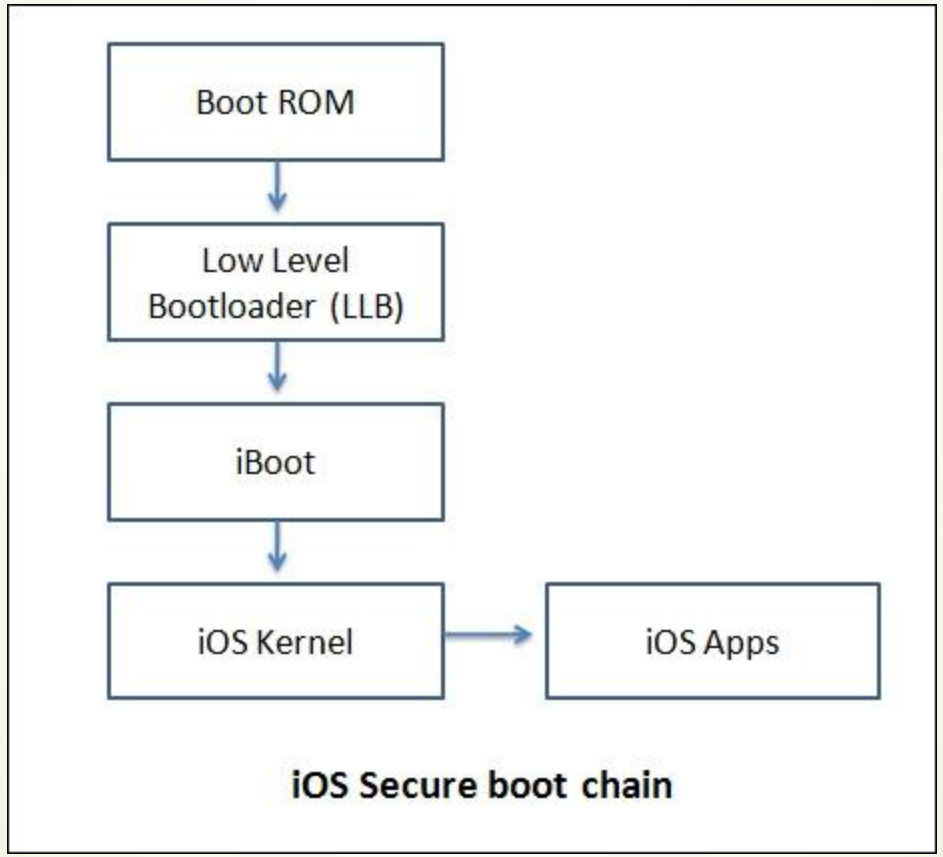
iOS Security Model

- The core security features of the iOS:
  - Secure boot chain
  - Code signing
  - Process-level sandboxing
  - Data-at-rest encryption
  - Native exploit mitigations: e.g. randomization in RAM

iOS Security Model- Secure boot chain

- A process by which the firmware is initialized and loaded on iOS devices at boot time– as first layer defence.
- In each boot step, each component is cryptographically signed by Apple and verified later to ensure that it has not been modified.
- Components- Boot ROM, low-level bootloader (LLB), iBoot image in flash memory, iOS kernel, .. then the iOS Apps.

iOS Security Model- Secure boot chain …

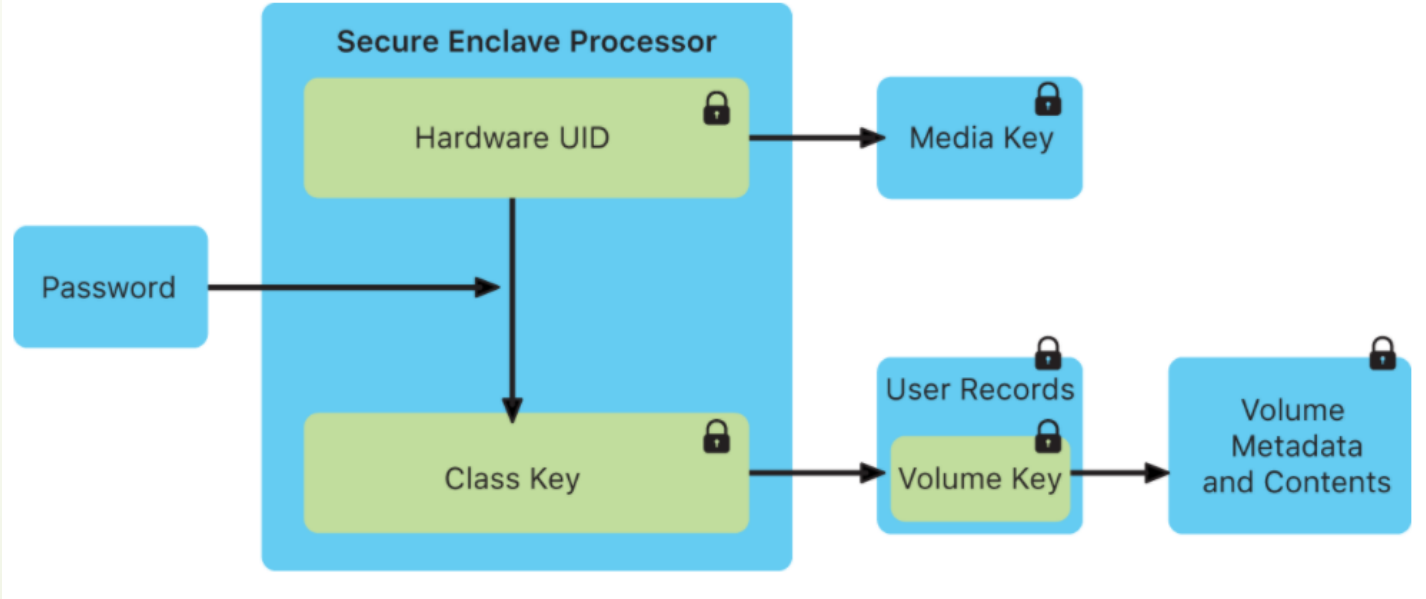


iOS Security Model- Secure enclave

- It is a separate processor and runs its own microkernel to handle your biometric information.
  - => not directly accessible by OS or any program running on a device.
- It provides all cryptographic operations for maintaining the integrity of biometric data- Touch ID and Face ID.
- E.g., Using biometric security as a car key.
- But, not used for- software update, device erasing, passcode settings, and configurations- instead passcode is used.

iOS Security Model- Secure enclave …

- The Secure Enclave processor



iOS Security Model- Touch ID

- Touch ID is fingerprint recognition feature designed by Apple.
- It has been part of iPhones since iPhone 5S until iPhone 8.
- iPads, and MacBook user account can have managed fingerprints using Face ID.
- As of iOS 8, it was integrated as a means of authentication to third-party app.
- In 2017 Apple released the iPhone 8 and 8 Plus with Touch ID, alongside the iPhone X which instead incorporated Face ID.
- In Touch ID management, apps have no direct access to the Secure Enclave or the registered fingerprints- use LocalAuthentication framework.

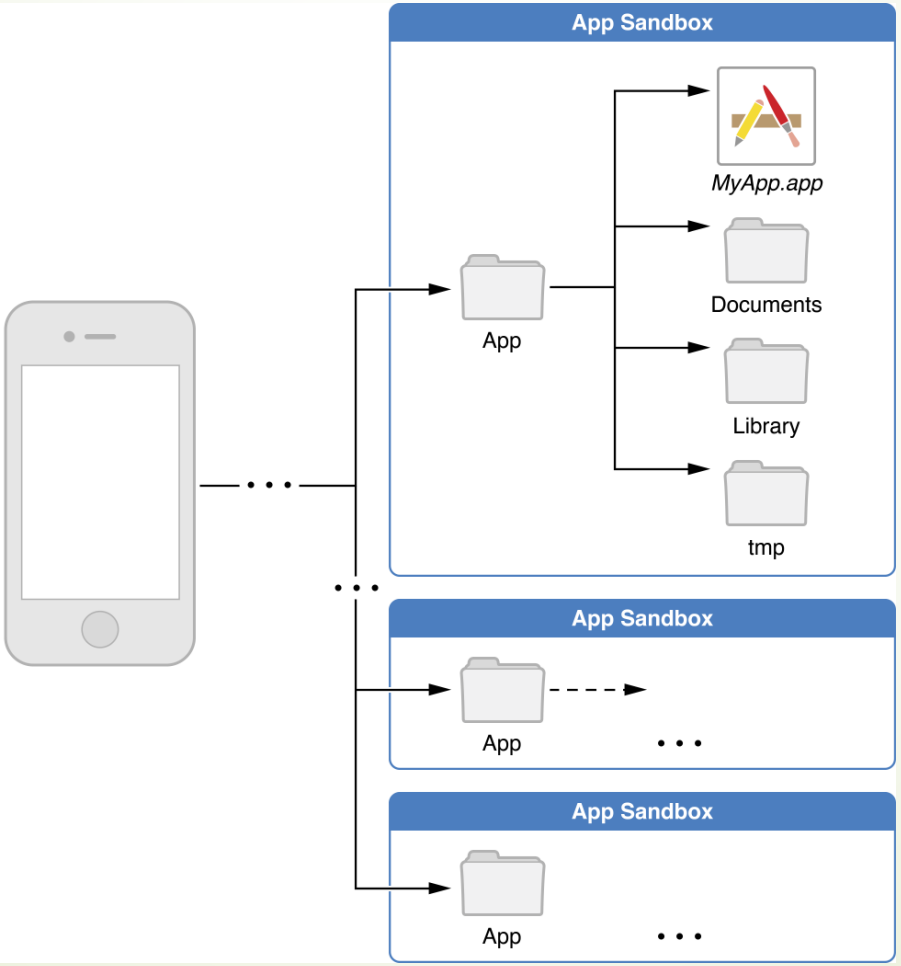
iOS Security Model- Code signing

- A runtime security feature that attempts to prevent unauthorized apps from running on the device
- It validates the app signature each time it is executed.
- For an app to run on iOS device, it must first be signed by a trusted certificate.
- Developers can install trusted certificates on a device through a provisioning profile signed by Apple.

iOS Security Model- Sandboxing

- All third-party applications on iOS run within a sandbox.
- It is a self-contained app environment that isolates from the OS and other apps.
- It introduces significant security to the platform and limits the damage that malware can do.
- This helps in the event that a malicious application has subverted the App Store review process.
- Each App is contained in its own directory and separation is maintained by XNU sandbox kernel extension (seat belt).

iOS Security Model- Sandboxing …



iOS Security Model- data-at-rest encryption

- By default, all data on the iOS is encrypted using block-based encryption (Advanced Encryption Standard- AES).
- It is encrypted with the file system key- generated on first boot and stored in block 1 of a NAND flash storage.
- The device uses this key during the startup process to decrypt the partition table and the system partition.
- The filesystem is encrypted only at rest.
- When the device is turned on, the hardware-based crypto accelerator unlocks the filesystem.
- This key helps for device's remote wipe capability as destroying the file system key causes the filesystem to become unreadable.

iOS Security Model- Exploit Mitigation Features

- What is an exploit?

An exploit is a piece of- software, a chunk of data, or a sequence of commands that takes advantage of a bug or vulnerability to cause unintended or unanticipated behavior to occur on computer software, hardware, or something electronic. Wikipedia

iOS Security Model- Exploit Mitigation Features …

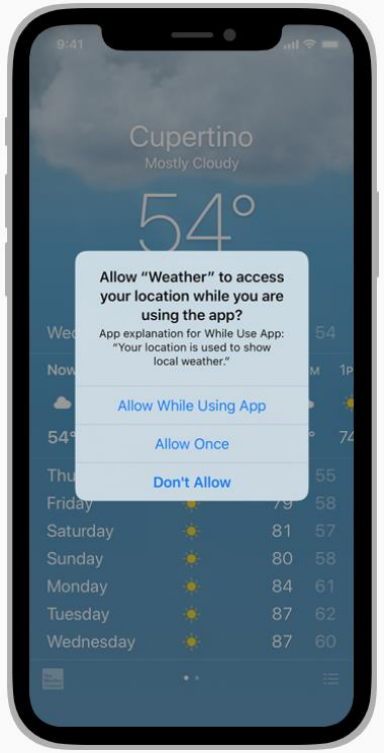
- iOS employs several exploit mitigation technologies to protect devices.
- ARM's Execute Never (XN) prohibit memory pages from writeable and executable at the same time- but can be bypassed using return-oriented programming (ROP).
- ROP controls the call stack, hijacks program control flow and execute chosen machine instruction sequences
- Address space layout randomization (ASLR) randomizes code locations- makes it complex for ROP to exploit memory corruption vulnerabilities.
- More on this- https://www.arm.com/blogs/blueprint/arm-exploit-mitigation

More on security features- App permissions

- The permission model on iOS segregates data in classes.
- Typically access classes - location services, contacts, calendar, photos, microphone, motion, Bluetooth, and social media data.
- When an app requires access to data protected by these privacy classes, it must prompt to allow or deny access.

More on security features- App permissions …

- Request for permission to …



More on security features- Data protection API

- It is the use of API that implements built-in hardware encryption to protect sensitive data stored on client-side.
  - fine-grained access control for user data stored in flash memory.
- The APIs are built on top of the Secure Enclave Processor (SEP)
- Data protection architecture overview

More on security features- iOS Keychain

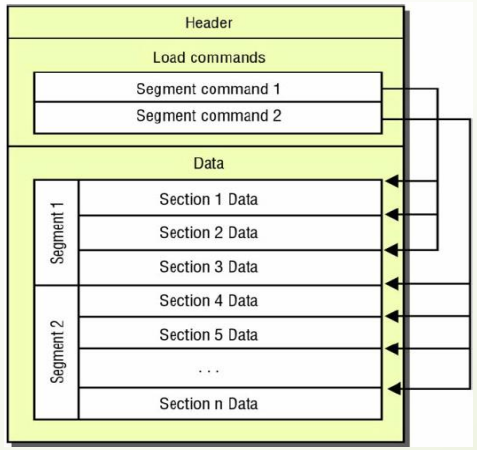
- iOS Keychain is a specialized database for storing metadata and sensitive information such as secrets and passwords.
- It is Apple's password management system where account names, passwords, and credit card numbers are securely stored and synced across all devices using iCloud.
- Technically, Keychain is highly secure in which the Keychain passwords and credit card numbers are encrypted with 256-bit AES.

Reverse engineering

- It is about how to get iOS binaries into a format one can work with, and how to identify some security-relevant features in these binaries.
- iOS apps compile to native code using the Mach-O (object) file format.
- Multiple Mach-O files (fat binaries) can be archived in one binary to provide support for different architectures- ARMv7, ARMv7s, ARM64.
- Apps downloaded from the App Store are encrypted and later decrypted at run time, on-device by the loader.

Reverse engineering …

- Components of the Mach-O file format:
  - Header- used to identify the file format
  - Load commands- specify location of symbol table, file's encrypted segments, names of shared libraries, etc.
  - Data- contain code or data that gets mapped to virtual memory by the dynamic linker.



Reverse engineering …

- Verify the presence of vulnerabilities using otool by inspecting the Mach-O header.
- Use otool to verify if stack smashing is enabled in iOS app- it protects exploits against stack overflow attacks.
  - The random value (canary) is placed on stack before local variables to protect saved base pointer, instruction pointer and function arguments.
  - The value of the canary is then verified when the function returns to see whether it has been overwritten.
- To reverse engineer Apps, you must disassemble and decompile your target application.. and inspect the features.
- Reverse engineering tools

Jailbreaking

- Jailbreaking- a process by which iOS is modified to remove restrictions and get greater user control over the device.
- The reason for jailbreaking is to install third-party applications that Apple couldn't approve in the App Store.
- Disadvantages- bricking, loss of warranty, vulnerable to malware, etc.
- Jailbreaking vs Unlocking- Jailbreaking removes software restrictions while unlocking removes restrictions imposed by cellular provider.
- It is possible to use iTunes and App Store after jailbreaking but some App Store apps include jailbreak detection which prevents running.

- **Does jailbreaking prevent from updating iOS?**

Defensive strategies to protect iOS apps

- Protecting data in iOS apps (more discussion on this).
- Avoiding injection vulnerabilities.
- Securing apps with binary protections.

Protecting data in iOS apps

- Avoid storing sensitive data in NSDefaults- consider UIWebView for suddenly stored data, etc.
- Plan about state transitions in the app and what data should be accessible in each state.
- Determine how access to the app will be protected. Example, use client-side authentication with Touch ID.
- Identify data entry points early and implement parse rules /validation along the way.
- Consider the impact of any third-party libraries that might be used within the application.

Guidelines for implementing encryption

- Use tried-and-tested encryption algorithm- e.g., use AES-XTS with a key size of 256 and hashing using SHA-256.
- Generate key using accepted key derivation function- e.g., keybag in Apple uses PBKDF2 with 10,000 iterations.
- The key used to encrypt data should itself be encrypted- also always keep the user input as large as possible.

Protecting data in transit

- Certificate pinning checks which **certificates** are considered valid for a particular website- limiting risk
- Pinning remains an important security control and is recommended for any application.
- This protection gives additional confidence that the host being connecting to is who it reports to be.
- It requires embedding a public key within an app that allows to compare it against one provided.

Issues for discussion (1)

- Why do enterprises employ their own distribution centers?
- What is a jailbreak? What are the advantages and disadvantages?
- Does jailbreaking prevent from updating iOS?
- What are the key features in iOS 14? Identify any three features in https://www.apple.com/ios/ios-14/features/ and discuss.
- Which of the following is not a high-level component of the iOS architecture? Cocoa touch, Media, Emulator, Core services , Core OS

Issues for discussion (2)

- (Protecting data in iOS apps) Watch the video (in link) on data encryption and decryption in Swift.
- Based on the video, write the object(s) relevant to the topics covered in this lecture- and discuss.
- Explore and provide a description of the RNCryptor object?

Issues for discussion (3)

- (Group work) Consider the following steps to make iOS Apps secure (described here).
  - Store confidential data in a secure place (1)
  - Make networking layer invulnerable (2)
  - Think about your secret (like API) keys (3)
  - Be careful with 3rd party integration (4)
  - Keep learning about new vulnerabilities (5)

Issues for discussion (3) …

- Steps to make iOS Apps secure …
- Each of the steps mentioned in previous slide are important security measures. However, ….
- Form a team, and then take any one of the steps (random picking).
- Discover from the Web and take note of list of argument points why the (random picked) step is more important than the other four.
- Present your work to classmates by posting your notes on menti.

Next topic

- Cross-platform mobile applications and security